W4 Building and training fuller comprehension + Deployment

Agenda:

Student-led clarification of things that are weird, don't make sense, etc.

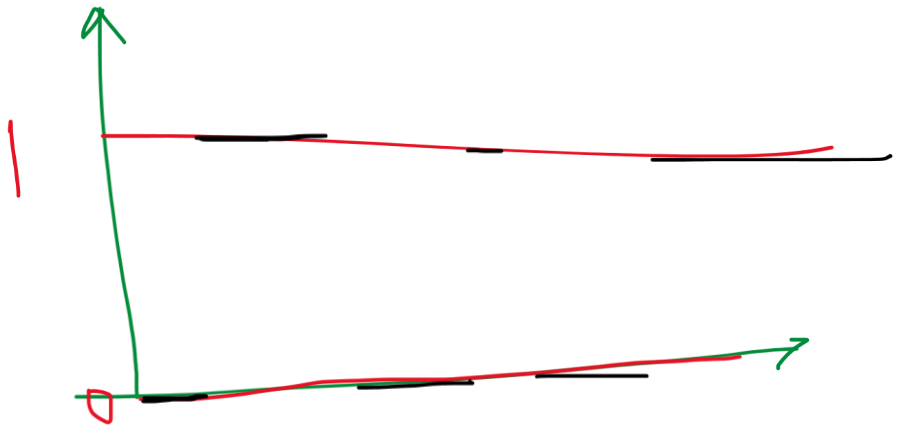
First, let's address the videos by Grant (3Blue1Brown).

What are your questions? What didn't make sense? What would you like to clarify further? (I'm just going to anticipate a few below, but you go ahead and lead with yours!)

- Around 9:20: what are the parameters we should want? → in the first layer, each neuron just captures the "color value" of the corresponding pixel. So its **activation** is just a value between 0 and 1. From the point of view of *any single neuron* in the second layer, what it gets is the sum of these activations from all the neurons in the first layer, but each is weighted with a particular weight (or importance). *These parameters are called weights: they are unknown, and the objective of the model is to find them*.
- What the heck is an activation, really? → it's just a number associated with a particular neuron, which you can think of in two useful ways: (1) as the neuron's output (it takes some information as input, and passes some information on as output), and (2) as the number that captures whether or not that neuron "fires"/"is activated", which means that it makes sense for it to be between 0 and 1
- What the heck is the sigmoid, and why do we use it again? → the sigmoid function is just some function, which takes an input and maps it onto the (0, 1) range. Because it makes sense to have each neuron's activation be in the (0, 1) range, we just use the Sigmoid function to remap the sum of all info arriving to the neuron into that range.
    - And what the heck is ReLU? And why are we using it instead?

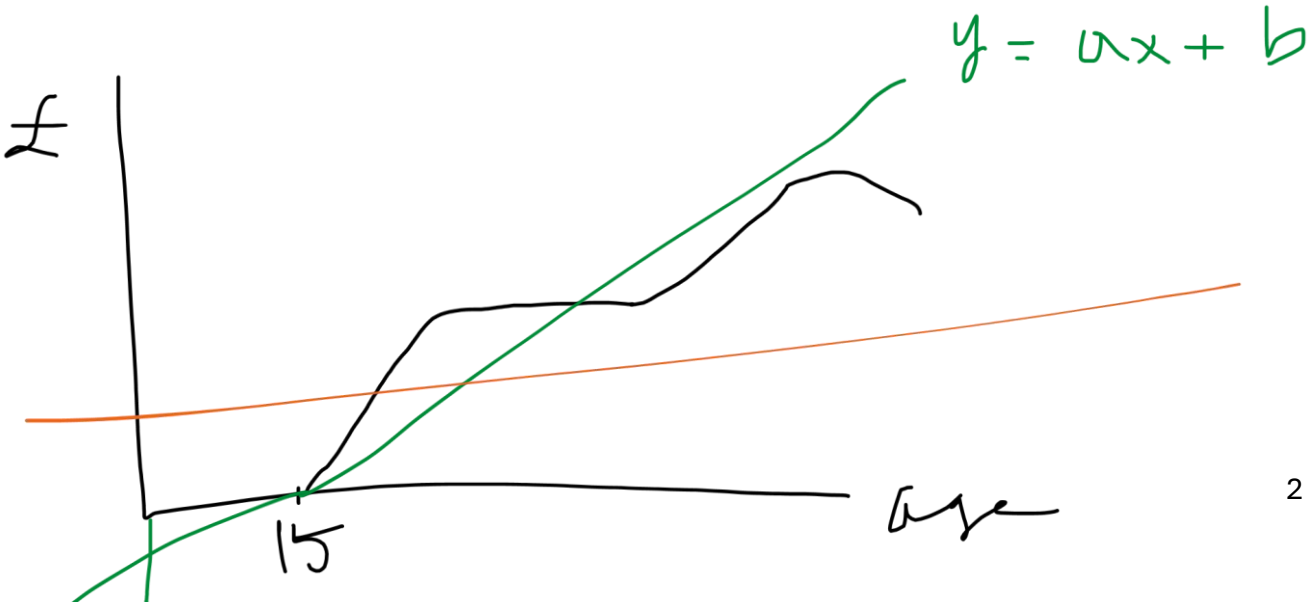
    - ReLU = max(0, a) → this just means, take the value of a and compare it to 0, and then go with whichever is largest.
    - This doesn't remap the activations to the (0,1) range, but it does eliminate all negative activations by setting them equal to 0.
- 12:40: learning = finding all the 13002 weights and biases. I would just like to contrast this to linear regression. We find (by hand) approximately 8-15 explanatory variables. Here though we find 13000…that's crazy.
- The whole network is just a function (though quite complicated)
- Cost function (=loss) as "mean square error"???
    - -> if you have an output for 10 digits, the output will consist of 10 numbers between 0 and 1. The "ideal" or correct output will be also 10 numbers, 9 of which are 0 (for those digits that aren't on the image) and 1 for the digit that is. For each digit, subtract the model's guess from the correct number for that digit (either 0 or 1) to find, for each digit, how far off the model's guess is. (We take the square of this in order to make all these distances be positive, so that they are easy to compare.) The cost function in this scenario is simply a function that takes all these distances (squared) and adds them up. C

Then: any other questions you have gathered? Things that don't make sense?



A linear functional is useful if:

- There is a **stable** relationship between two variables (stable = unchanging across all the values of the variables)



And now, a reference: Michael Nielsen's online book *Neural Networks and Deep Learning*: helpful and (but) very detailed. I don't really share his ideas about intelligence, for reasons we can discuss, but it's a very helpful explanation of backpropagation etc.

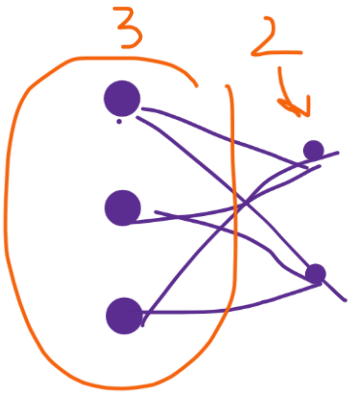
Fuller understanding of model structure and building it

Building Models YT tutorial (how layers work + some special functions)

Fuller description of **how the layers work**, what layers really are (functions for transforming incoming information), and what data manipulation layers are

- Convolutional layer
- Linear layer
- (Skip RNN and Transformers for now)
- Max pooling and min pooling – but why do we do this
- Activation functions

Docs on torch.nn :
include useful things, such as the description that each layer's name simply comes from the mathematical function that its activation (towards the next layer) represents, see eg linear layer: https://docs.pytorch.org/docs/stable/nn.html#transformer-layers



Covering a 3x3 image with a 2x2 window gives us 4 options (2x2 options)

Training Models YT tutorial (data handling, loss/optimizer functions, and full training)

1. Fuller description of the **data handling** (incl. `Dataset` and `DataLoader` classes)

2. Fuller description of **loss functions** and **optimizers**

3. Putting it all together into training
    a. Reference TensorBoard, which I don't think I'll touch on, which is helpful for monitoring training progress
    b. Also includes a demo of saving a model – pay attention to it being saved in VS code / your working folder. Why are there many different versions saved?

Understanding Backpropagation and PyTorch's autograd (may not get to this)

Backpropagation
= algorithm for computing the gradient
Autograd helps us do this.

The Autograd tutorial is a little confusing, as it has two parts, and while the second is about how Autograd works in models, the first tries to explain the "back-end" a bit more theoretically.

Explanation for backpropagation:

- We start at the output layer, with a specific node: during training, given a particular output (guess) for that node, cost function tells us how far it is from the ideal (correct) number
- How do we get this node to be closer to the correct number? It has to do with the information it receives, which has the following components:
    - Activations (numbers) of all nodes in the previous layer → we don't control this (hold on…)
    - Weights for each node → we can adjust this
    - Some squishification (sigmoid/ReLU), which doesn't matter (kind of acts like a constant, so we don't tweak it)
- So the question is, in what direction do we want to nudge the weights that belong to the nodes in the previous layer? = calculate gradients (in PyTorch: `.backwards()`)
- Having decided that, we turn to the activations of the nodes in the previous layer. What determines these? Well, it's the activations of the previous-previous layer nodes, weighted with their weights… = the same thing once again, calculate gradients
- So we just go through the network layer by layer, starting from all individual nodes in the output layer, going backwards layer by layer, calculating how to adjust the weights

3Blue1Brown video on backpropagation:
https://www.3blue1brown.com/?v=backpropagation
Watch this at home after this session please!

Saving and Loading models – may not have time for this

Using the Saving and Loading models PyTorch tutorial
(please also refer to What is a state_dict in PyTorch? reference)

We don't want to train and retrain models all the time... the goal is to train them, save them, and then use them for inference (→ Deployment).

For inference:
Remember that you must call `model.eval()` to set dropout and batch normalization layers to evaluation mode before running inference. Failing to do this will yield inconsistent inference results.

There are two ways to save a model—saving the entire model, or only saving its state_dict object, which is the recommended way. I'll give you both (→ .ipynb file), but include only the recommended one here in this file.

Save:

```
torch.save(model.state_dict(), PATH)
```

Load:

```
model = TheModelClass(*args, **kwargs)

model.load_state_dict(torch.load(PATH, weights_only=True))

model.eval()
```

What goes in PATH is the filepath where you save your model/file name.
A common PyTorch convention is to save models using either a `.pt` or `.pth` file extension.

There are multiple other considerations for how to load/save a model using different parameters, or moving from GPU to CPU etc, but I won't walk you through those. These are available in the .ipynb file.